U A D E
UNIVERSIDAD ARGENTINA DE LA EMPRESA · Facultad de Ingeniería y Ciencias Exactas

Guía para la presentación

MediConecta · Entrega Parcial N.º 2 · Mensajería asincrónica

Trabajo Práctico: Desarrollo de Aplicaciones II
Comisión Lunes TT · Prof. Matías Ruiz
Grupo 4: Agustín Bodas, Máximo Bonarrico y Sergio Casanova
19 de octubre de 2026

Para qué sirve esta guía

Es el libreto del grupo para mañana: qué preparar antes, en qué orden mostrar las cosas, qué decir en cada diapositiva, qué partes del código abrir frente al profesor y cómo responder las preguntas más probables. Está pensada para leerla una vez entera y tenerla a mano durante la presentación.

Lo que pidió el profesor y cómo lo cubrimos.

- **"Usar la mayor cantidad de patrones de diseño posible".** Hoy hay 13 patrones con código real, no solo nombrados (diapositiva 10). El recorrido por el código de la sección 4 los muestra encadenados en un mismo flujo, que es lo que más impresiona.
- **"Agregar cosas fuera del marco del ejemplo".** El **Monitor de eventos**, una pantalla que muestra en vivo qué suscriptor procesó cada evento y cuánto tardó; bandeja de notificaciones con contador en vivo, auditoría como segundo suscriptor, pruebas de fallas (rollback, duplicados, caída, DLQ), optimización de rendimiento (4 veces más rápida), prueba automática de punta a punta y diez diagramas con su documento maestro. La sección 7 propone qué más sumar para el 09/11.
- **"Indagar más en el código durante la presentación".** La sección 4 es un recorrido de diez paradas por el código, con archivo y línea, y la sección 5 tiene dos demostraciones en vivo con la consola de WildFly que muestran el broker por dentro.

La objeción más probable: "¿no es la misma demo que la Entrega 1?". La pantalla del paciente es la misma a propósito: el proceso asincrónico se engancha a la operación que ya existía. Lo nuevo es todo lo que pasa *después* del clic. Por eso la presentación abre con la diapositiva comparativa (2) y la demo se hace con la profesional parada en el **Monitor de eventos** y la consola del broker a la vista, no en la agenda.

1. Antes de entrar al aula

☐ Abrir el panel de Supabase y confirmar que el proyecto **no está pausado**. El plan gratuito lo pausa después de una semana sin uso; si pasa, WildFly arranca con el error `tenant/user ... not found`.

☐ Compilar y desplegar: `mvn clean package` y copiar `target/mediconecta.war` a `standalone/deployments` de WildFly.

☐ Levantar con `./wildfly/iniciar-servidor.sh` (usa el perfil `standalone-full.xml`, que trae el broker). Esperar el mensaje `WFLYSRV0025 ... started` y verificar en el log las dos líneas `Started message driven bean`.

☐ Abrir dos ventanas del navegador, una normal y otra de incógnito, para tener dos sesiones a la vez: paciente `juan.perez@mediconecta.test` y profesional `ana.lopez@mediconecta.test`, ambos con contraseña `demo1234`. La de la profesional, parada en **Monitor de eventos**.

☐ Con la profesional, publicar uno o dos horarios con fecha futura antes de empezar, para no perder tiempo en la demo.

- [ ] Tener el IDE abierto con los archivos de la sección 4 en pestañas, en ese orden.
- [ ] Tener una terminal lista con `jboss-cli.sh --connect` para las demostraciones de la sección 5.
- [ ] Llevar como respaldo el PDF de la presentación (`docs/presentacion/MediConecta_Entrega2.pdf`), por si la compu del aula no tiene PowerPoint.

> **Plan B si algo no levanta.** Si Supabase no responde o no hay red, no improvisen. Muestren las diapositivas 5 a 7, que tienen capturas reales de la app funcionando, y pasen directo al recorrido por el código. Todo lo que se muestra en el código existe y se puede abrir sin necesidad de que la app esté corriendo.

2. Reparto sugerido

Unos quince minutos en total. Cada uno presenta una parte, pero los tres tienen que poder contestar sobre cualquiera, porque la consigna evalúa la comprensión individual.

Integrante	Diapositivas	Qué le toca
Máximo Bonarrico	1 a 7	Apertura, comparación con la Entrega 1, qué sumamos, el flujo y la demo en vivo con el Monitor de eventos
Agustín Bodas	8 y 9	Sincrónico frente a asincrónico, y las garantías ante fallas, con la demo de suscriptor caído desde la consola
Sergio Casanova	10 a 13	Patrones, extras, lo que viene y la apertura del código

Es una sugerencia: lo importante es que cada uno ensaye su parte en voz alta al menos una vez y que los tres lean la sección de preguntas.

3. Guion diapositiva por diapositiva

#	Min.	Diapositiva	Qué decir (las notas del orador de la PPTX tienen el detalle)
1	0:30	Portada	Qué es MediConecta en una frase y que esta entrega suma la primera integración asincrónica real, con broker configurado.
2	1:00	Entrega 1 vs. 2	Anticipar la objeción: la pantalla es la misma, lo que pasa después del clic no. Leer la tabla de arriba a abajo.
3	0:45	Qué sumamos	Los cuatro puntos. El broker no es simulado: es ActiveMQ Artemis dentro de WildFly.
4	1:30	El flujo	La caja punteada es una sola transacción: si el turno no se guarda, el mensaje no sale. A la derecha, dos suscriptores que no se conocen.
5–7	4:00	Demo	En vivo, con el Monitor de eventos (ver abajo). Las capturas quedan de respaldo.
8	1:15	Sincrónico vs. asincrónico	La regla: si el usuario necesita la respuesta para seguir, sincrónico; si puede esperar o tiene varios interesados, asincrónico.
9	1:15	Fallas	Los cuatro casos, todos probados. El del suscriptor caído ya se vio en vivo en la demo.
10	1:30	Patrones	Los 13, y sobre todo que están encadenados (ver la parada 1 del recorrido).
11	1:00	Además de lo pedido	Rendimiento, pruebas automáticas, diagramas y el bug de login corregido.
12	0:45	Lo que viene	Ya está en el diagrama de integración con línea punteada.
13	—	Abramos el código	Invitar al profesor a elegir qué abrir; si no elige, seguir la sección 4.

La demo en vivo

Tres cosas a la vista desde el principio, si se puede en la misma pantalla: la ventana del paciente, la de la profesional abierta en **Monitor de eventos** y una terminal con `jboss-cli.sh --connect`.

1. **Paciente:** en Agenda, elegir a la Dra. Ana Lopez, “Ver turnos disponibles” y “Reservar”. Aparece el cartel ámbar del hold: es el EJB `@Stateful` con `@StatefulTimeout` de 5 minutos.
2. Tocar “Confirmar turno”. Responde en un segundo y medio, más o menos.
3. **Monitor de la profesional**, sin tocar nada: en menos de tres segundos aparece una fila “Confirmado” con ✓ **y los milisegundos** de Notificaciones y de Auditoría. Decir: “esto no existía en la Entrega 1: es el broker trabajando”.
4. **Consola:** frenar el suscriptor de auditoría:

```
/deployment=mediconecta.war/subsystem=ejb3/message-driven-
bean=AuditoriaSubscriber:stop-delivery
```

5. **Paciente:** cancelar ese turno. En el monitor: Notificaciones ✓ y Auditoría **Pendiente**; el contador de “entregas pendientes” sube a 1. Si quieren, mostrar también que el broker lo tiene retenido:

   ```
   /subsystem=messaging-activemq/server=default/jms-topic=turnos:count-messages-for-subscription(client-id=mediconecta-auditoria,subscription-name=auditoria)
   ```

6. **Consola:** reactivar, y en el monitor Auditoría pasa sola a ✓, con una latencia de varios segundos: es el tiempo que el broker guardó el mensaje.

   ```
   /deployment=mediconecta.war/subsystem=ejb3/message-driven-bean=AuditoriaSubscriber:start-delivery
   ```

7. Cerrar mostrando la bandeja de Notificaciones de la profesional, con los avisos y el contador del menú.

Opcional: en el log de WildFly, el productor aparece en el hilo del request (`default task-N`) y los consumidores en hilos del broker (`activemq-client-global`).

```
grep -E "encolado en topico|SMTP simulado|Auditoría\]" <WILDFLY_HOME>/standalone/log/server.log | tail -6
```

4. Recorrido por el código

Este es el corazón de la presentación, porque es lo que el profesor pidió explícitamente. El orden sigue el viaje de un turno confirmado, de la pantalla a la base, así cada parada se apoya en la anterior. En cada una hay qué mostrar y qué decir; las líneas corresponden al código actual.

Parada 1 · La fachada que coordina todo

`negocio/ServicioDeTurnosBean.java · líneas 39–40 y 93–107`

Mostrar `@Stateful` y `@StatefulTimeout` arriba de la clase, y después el método `confirmar()`. Decir: "Este método es una fachada. La pantalla hace una sola llamada y acá adentro se busca el turno, se cambia su estado, se guarda y se publica el evento, todo en una única transacción que abre el contenedor. Además, esta clase es el *director* del Builder: arma el evento paso a paso". Señalar `@RolesAllowed("PACIENTE")` en la línea 93: seguridad declarativa.

Parada 2 · El hold que se libera solo

`negocio/ServicioDeTurnosBean.java · líneas 156–167`

Mostrar `@PreDestroy`. Decir: "Si el paciente no confirma en cinco minutos, el contenedor destruye la conversación y, antes, llama a este método, que devuelve el turno a disponible. No escribimos ningún temporizador: el ciclo de vida lo maneja el contenedor EJB".

Parada 3 · El Builder del evento

`mensajeria/EventoTurno.java · líneas 51–68`

Mostrar `build()`. Decir: "El builder valida lo obligatorio y siempre genera el `idEvento` con un UUID. Ese identificador es la clave de la idempotencia, así que el builder hace imposible publicar un evento sin él. Fíjense que el evento no lleva ningún dato clínico: es minimización de datos, porque la historia clínica es un dato sensible (Ley 25.326)".

Parada 4 · Publicar dentro de la transacción

`mensajeria/PublicadorEventosTurno.java · líneas 30 y 38–47`

Mostrar el `JMSContext` inyectado. Decir: "Este contexto usa la connection factory XA de WildFly, así que el envío se enlista en la transacción del que llama. El mensaje queda pendiente y sale recién con el commit. Es el patrón Transactional Client. La propiedad `tipoEvento` nos permite, más adelante, que facturación filtre solo los eventos que le interesan. Y en la misma transacción se guarda el evento en `evento_publicado`, que es lo que después le permite al monitor saber qué suscriptor tiene algo pendiente".

Parada 5 · El consumidor durable

`mensajeria/NotificacionesSubscriber.java · líneas 24–30 y 44–52`

Mostrar la anotación `@MessageDriven` con su `activationConfig`. Decir: "Suscripción durable con nombre y `clientId`: si este componente está caído, el broker le guarda los mensajes. El `maxSession` en 1 es porque una suscripción durable con `clientId` admite una sola conexión". Después mostrar `onMessage`: primero registra el evento como procesado y después recorre los procesadores con `Instance<ProcesadorNotificacion>`, es decir, CDI descubre solo todas las implementaciones.

Parada 6 · Template Method y Factory Method

`notificaciones/ProcesadorNotificacion.java · líneas 25–37 · ProcesadorEmail.java · ProcesadorAvisoProfesional.java`

Mostrar que `procesar()` es `final`: fija el orden (crear, enviar, guardar). Las subclases solo redefinen los pasos. Decir: "Sumar SMS es agregar una subclase, sin tocar el MDB". Pregunta trampa posible: ¿por qué es `@Dependent`? Porque los beans con scope normal (`@ApplicationScoped`) usan un proxy, y un proxy no puede interceptar métodos `final`; Weld rechaza el despliegue.

Parada 7 · Idempotencia de verdad

`dao/EventoProcesadoDAO.java · líneas 21–26`

Decir: "Primero buscamos si el evento ya se procesó; si sí, se descarta sin romper la transacción. Pero la consulta sola no alcanza: dos entregas simultáneas podrían pasar la consulta a la vez. Por eso la tabla tiene la clave `suscriptor:idEvento` como primaria: la segunda falla al commitear, el broker la reentrega y ahí se descarta".

Parada 8 · El broker configurado

`wildfly/configurar-mensajeria.cli · líneas 18 y 21`

Mostrar la creación del tópico y el `address-setting`: tres intentos, cinco segundos entre cada uno, y después a la DLQ. Decir: "Esto es lo que pide la consigna con 'broker configurado': no es un mock".

Parada 9 · Fuera del marco: rendimiento y un bug

`seguridad/MecanismoAutenticacion.java · líneas 50–59 · web/LoginBean.java · línea 90 · web/AgendaBean.java · líneas 61–73`

Decir: "Medimos que cada página tardaba 3,4 segundos. Encontramos que el mecanismo de autenticación consultaba la base en cada request, incluso para los CSS y los JS, y que JSF evaluaba los getters varias veces por render. Lo resolvimos guardando el resultado durante el request, y bajó a 0,8 segundos. Lo delicado fue `getMisTurnos`: la tabla lee la lista antes de ejecutar la acción, así que hay que invalidarla al confirmar o cancelar". También mostrar el `SEND_FAILURE` de la línea 52, que corrige el error "Unauthorized" ante una contraseña incorrecta.

Parada 10 · El monitor: Transfer Object y seguridad por titularidad

`negocio/ServicioDeMonitoreoBean.java · líneas 35, 54 y 63–77 · negocio/dto/FilaMonitorDTO.java`

Decir: "El monitor cruza tres tablas: los eventos publicados, lo que registró cada suscriptor y el usuario. A la vista no le pasamos entidades sino Transfer Objects con solo lo que muestra. Y fíjense la línea 54: el usuario no viene de un parámetro sino del principal autenticado, con `SessionContext`. `@RolesAllowed` decide quién entra; esto decide qué ve cada uno. Así nadie puede pedir los eventos de otro cambiando un id". La latencia es la diferencia entre `publicado_en` y `procesado_en` (línea 77).

Si el profesor pide ver las pruebas. Está en el repositorio: `pruebas/prueba_e2e.py` recorre la app con un navegador real y hace 14 verificaciones, y `pruebas/prueba_monitor.py` hace otras 7 sobre el monitor, incluido el escenario del suscriptor frenado desde la consola. Se corren con `python pruebas/<archivo>` (requieren `pip install playwright` y Chrome).

5. Demostraciones con la consola de WildFly

La demostración A ya está integrada en la demo en vivo, con el monitor. Acá quedan los comandos sueltos por si el profesor pide repetirla o verla desde la consola.

A. Suscriptor caído: el mensaje no se pierde

Frenar la entrega al suscriptor de auditoría:

```
/deployment=mediconecta.war/subsystem=ejb3/message-driven-bean=AuditoriaSubscriber:stop-delivery
```

Confirmar un turno desde la app. Las notificaciones llegan igual, pero la auditoría no. Ver el mensaje retenido en el broker (tiene que dar `1L`):

```
/subsystem=messaging-activemq/server=default/jms-topic=turnos:count-messages-for-subscription(client-id=mediconecta-auditoria,subscription-name=auditoria)
```

Reanudar la entrega y repetir el conteo, que vuelve a 0. En el log aparece la línea de auditoría del evento retenido:

```
/deployment=mediconecta.war/subsystem=ejb3/message-driven-bean=AuditoriaSubscriber:start-delivery
```

Lo probamos incluso reiniciando WildFly entero en el medio, y el mensaje también sobrevivió.

B. La cola de mensajes fallidos

```
/subsystem=messaging-activemq/server=default/jms-queue=DLQ:count-messages
```

Debería dar 0. Sirve para explicar el Dead Letter Channel aunque no haya mensajes rotos: si un mensaje fallara tres veces, quedaría ahí sin trabar al resto.

6. Preguntas probables y cómo contestarlas

¿No es la misma demo que la Entrega 1?

La pantalla del paciente sí, a propósito: el proceso se engancha a la operación que ya existía. Pero en la Entrega 1 el clic terminaba en un UPDATE; ahora publica un evento en un broker real, lo procesan dos componentes independientes, le llega el aviso a otro usuario y el monitor muestra cuánto tardó cada uno. Y si frenamos un suscriptor, el mensaje no se pierde. Nada de eso existía.

¿Para qué guardan `evento_publicado` si el mensaje ya está en el broker?

Porque la aplicación no puede leer la suscripción de otro sin consumirla. Guardar el evento en la misma transacción en la que se envía nos da un registro confiable de lo publicado, y cruzándolo con `evento_procesado` sabemos qué falta procesar. Es la misma idea del patrón *Transactional Outbox*, usada acá para observabilidad.

¿Por qué un tópico y no una cola para las notificaciones?

Porque el mismo evento le interesa a más de un componente (notificaciones y auditoría hoy, facturación después) y cada uno hace algo distinto. Una cola entrega cada mensaje a un solo consumidor; un tópico se lo entrega a todos los suscriptores, y el productor no tiene que conocerlos.

¿Qué pasa si se cae el broker?

Los mensajes son persistentes: Artemis los guarda en su journal en disco. Al volver, entrega lo pendiente. Lo verificamos reiniciando WildFly con un mensaje retenido.

¿Qué pasa si algo falla después de enviar el mensaje pero antes del commit?

La transacción hace rollback y el mensaje se descarta, porque el envío estaba enlistado en ella. Nadie recibe un aviso de un turno que no se guardó. También lo probamos.

¿La base de datos participa en una transacción XA?

Respuesta honesta: no. El datasource de PostgreSQL no es XA; el broker sí. El gestor de transacciones de WildFly (Narayana) admite un único recurso no XA en una transacción distribuida, y lo confirma último (*Last Resource Commit Optimization*). Para esta entrega alcanza; para un entorno real convendría un datasource XA o el patrón *Transactional Outbox*.

¿Por qué el servicio de turnos es stateful y no guardan el hold en la base?

Porque el hold es parte de una conversación con un paciente puntual y vive lo que dura su decisión. Con `@Stateful` y `@StatefulTimeout` el contenedor libera el turno solo cuando vence el plazo. Igual, el estado HOLD también se guarda en el turno para que nadie más lo vea disponible.

¿Por qué los MDB no usan `@RunAs`?

Porque llaman a DAOs, que no tienen restricciones de rol. Si un MDB tuviera que invocar un EJB con `@RolesAllowed`, necesitaría `@RunAs("SISTEMA")`, como está previsto en el documento de diseño para facturación.

¿Por qué JSON y no un ObjectMessage con el objeto Java?

Porque desacopla: el consumidor no necesita las mismas clases que el productor, otro componente hecho en otra tecnología podría consumirlo, y se evita la deserialización de objetos Java, que es un riesgo de seguridad conocido. Además se lee en los logs.

¿El email se envía de verdad?

No: el envío SMTP está simulado y se registra en el log como `[SMTP simulado]`. Lo que se evalúa en esta entrega es la mensajería, y eso sí es real. Conectar un SMTP es cambiar una sola clase (`ProcesadorEmail.enviar`).

¿Cómo escalarían el consumo si hubiera muchos eventos?

Hoy cada suscripción durable usa una sola sesión. Para escalar se usan suscripciones compartidas de JMS 2.0, o, para tareas que deben hacerse una sola vez, una cola con varios consumidores en competencia (*Competing Consumers*), que es lo que planificamos para las autorizaciones.

¿Qué cambiarían si empezaran de nuevo?

Pondríamos la memoización por request desde el principio, porque el problema de rendimiento estuvo oculto hasta que medimos, y definiríamos antes la matriz de roles completa, incluido el administrador de clínica.

7. Ideas para sumar antes del 09/11

Pensadas en lo que pidió el profesor: más patrones y funcionalidades que vayan más allá del ejemplo, aprovechando lo que ya está construido.

Idea	Patrón que suma	Por qué vale la pena
Lista de espera: al cancelarse un turno, avisar al primero en espera	Strategy (orden de llegada o prioridad), tercer suscriptor del tópico	Funcionalidad fuera del ejemplo que reutiliza la mensajería: no toca el servicio de turnos.
Recordatorio automático 24 horas antes del turno	Timer de EJB (`@Schedule`) y Singleton	Muestra otro servicio del contenedor y genera eventos nuevos.
Estados del turno como clases	State	Hoy es un enum; con State, transiciones inválidas como “cancelar un turno atendido” se vuelven imposibles por diseño.
Validaciones antes de reservar (superposición de turnos, afiliado activo)	Chain of Responsibility	Cada regla es un eslabón; sumar una no toca el servicio.
Integración con la obra social	Adapter/Gateway y Circuit Breaker	Ya es obligatorio para el 09/11 (SOAP); con el breaker se suma el desafío opcional de resiliencia.
Reprogramar un turno	Nuevo tipo de evento con el Builder existente	La consigna menciona la reprogramación y todavía no la tenemos.
Temas claro y oscuro	—	Puntos extra explícitos del anexo.

Responsive (obligatorio del anexo): resuelto. En pantallas de menos de 900 píxeles la barra lateral pasa arriba como un menú desplazable y las tablas se desplazan dentro de su tarjeta. Si el profesor achica la ventana o la abre en el celular, se adapta. Probado a 390, 820 y 1366 píxeles de ancho sin desborde horizontal.